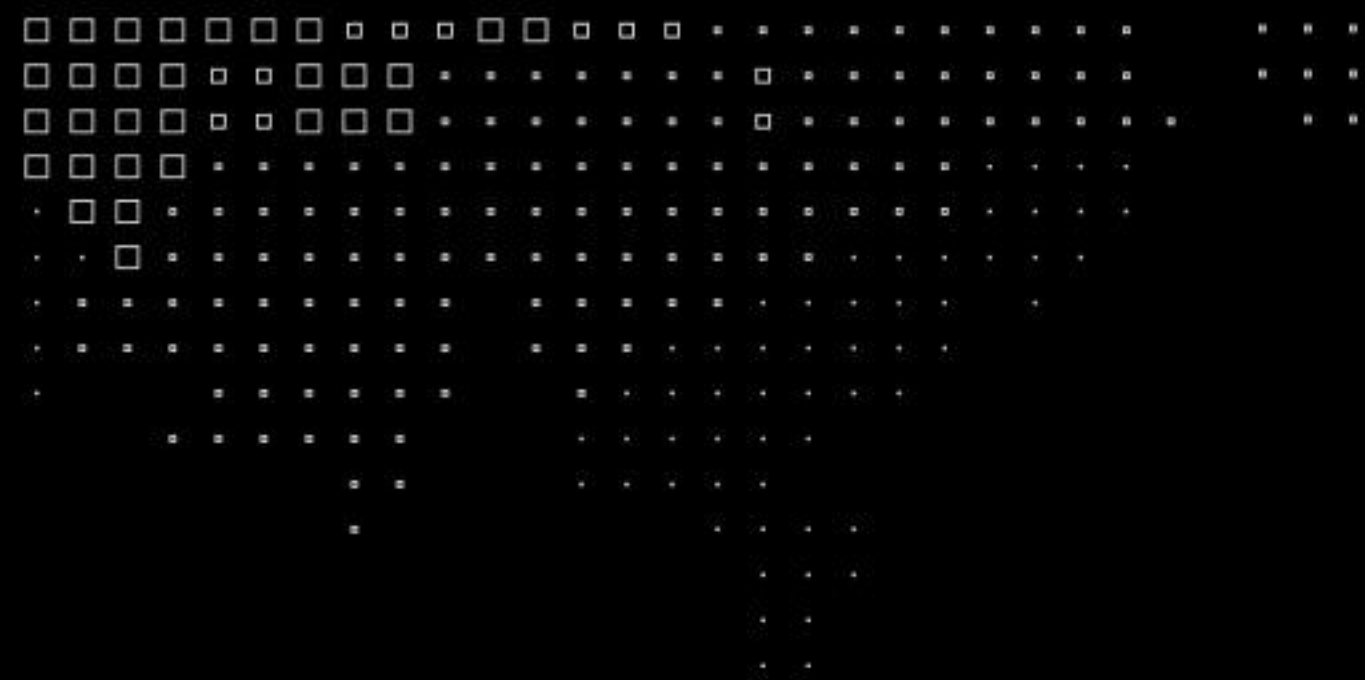
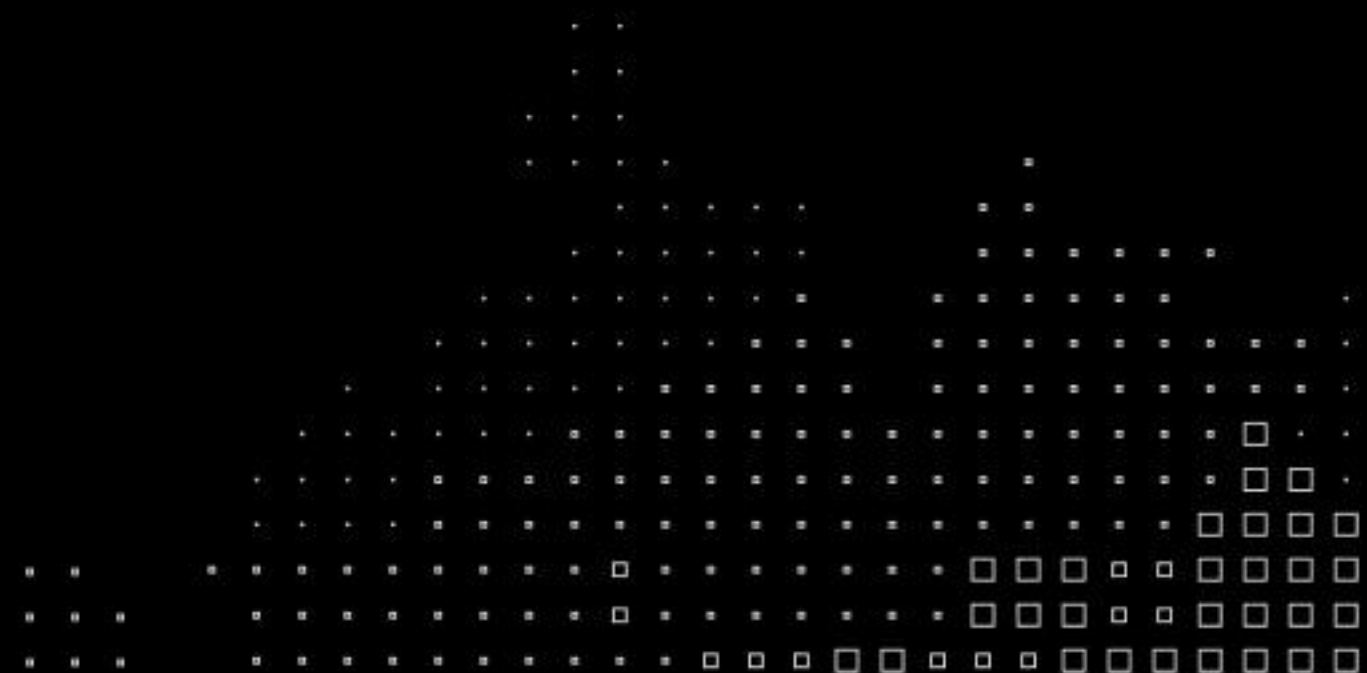




Ciência da Computação



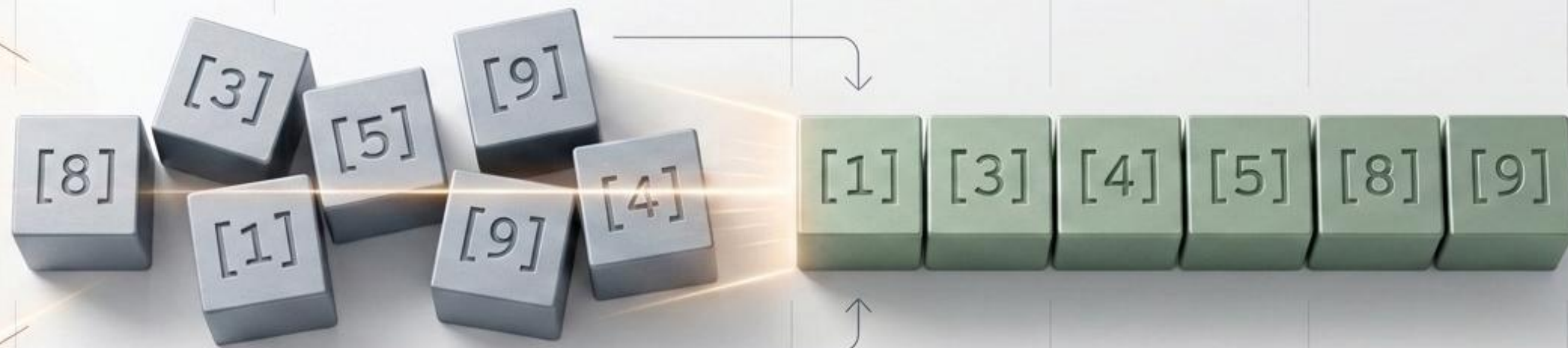
DSA

Data Structures and Algorithms

Professor Álvaro Gonçalves

Email: profalvaro.goncalves@fiap.com.br

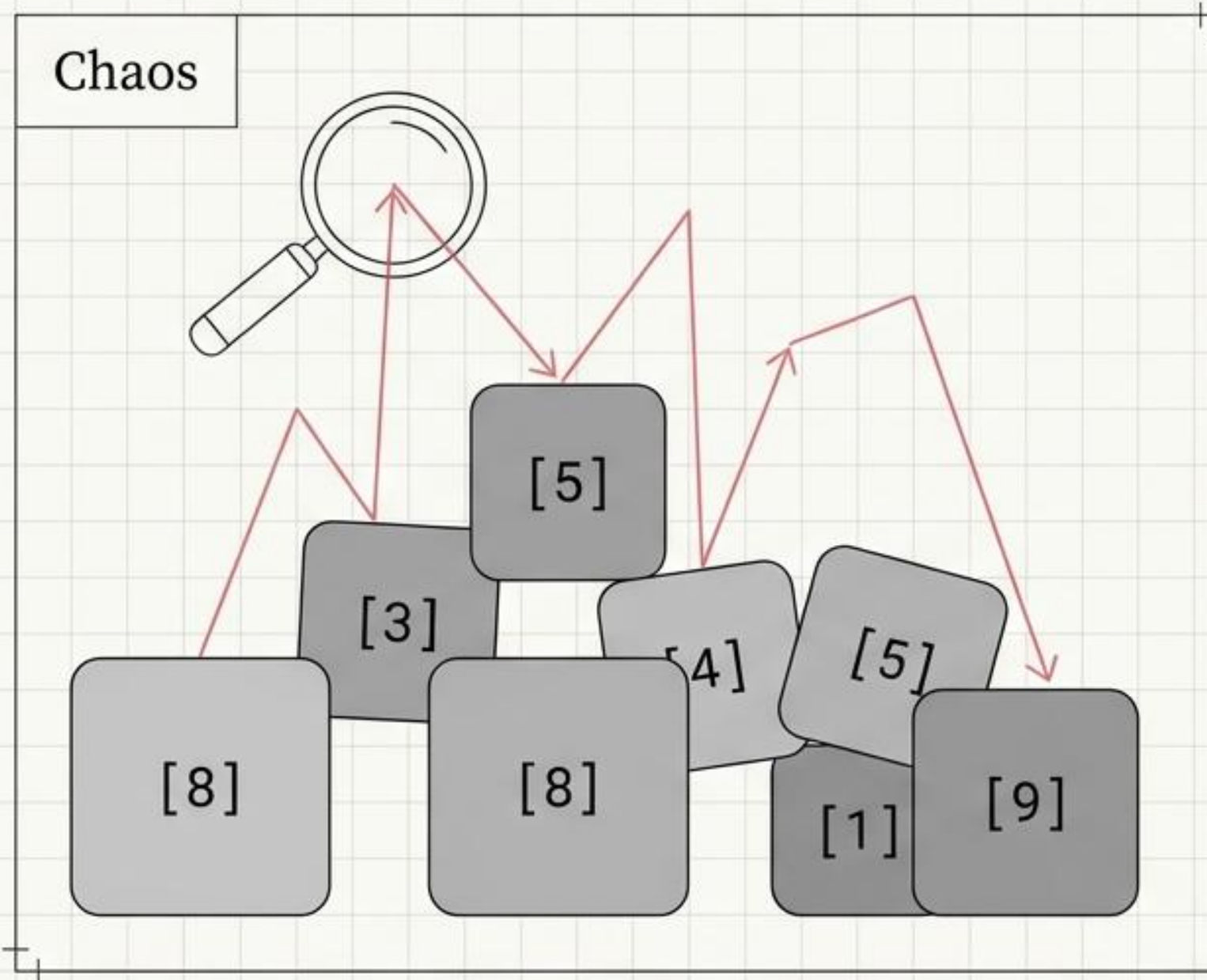
A Oficina da Ordenação



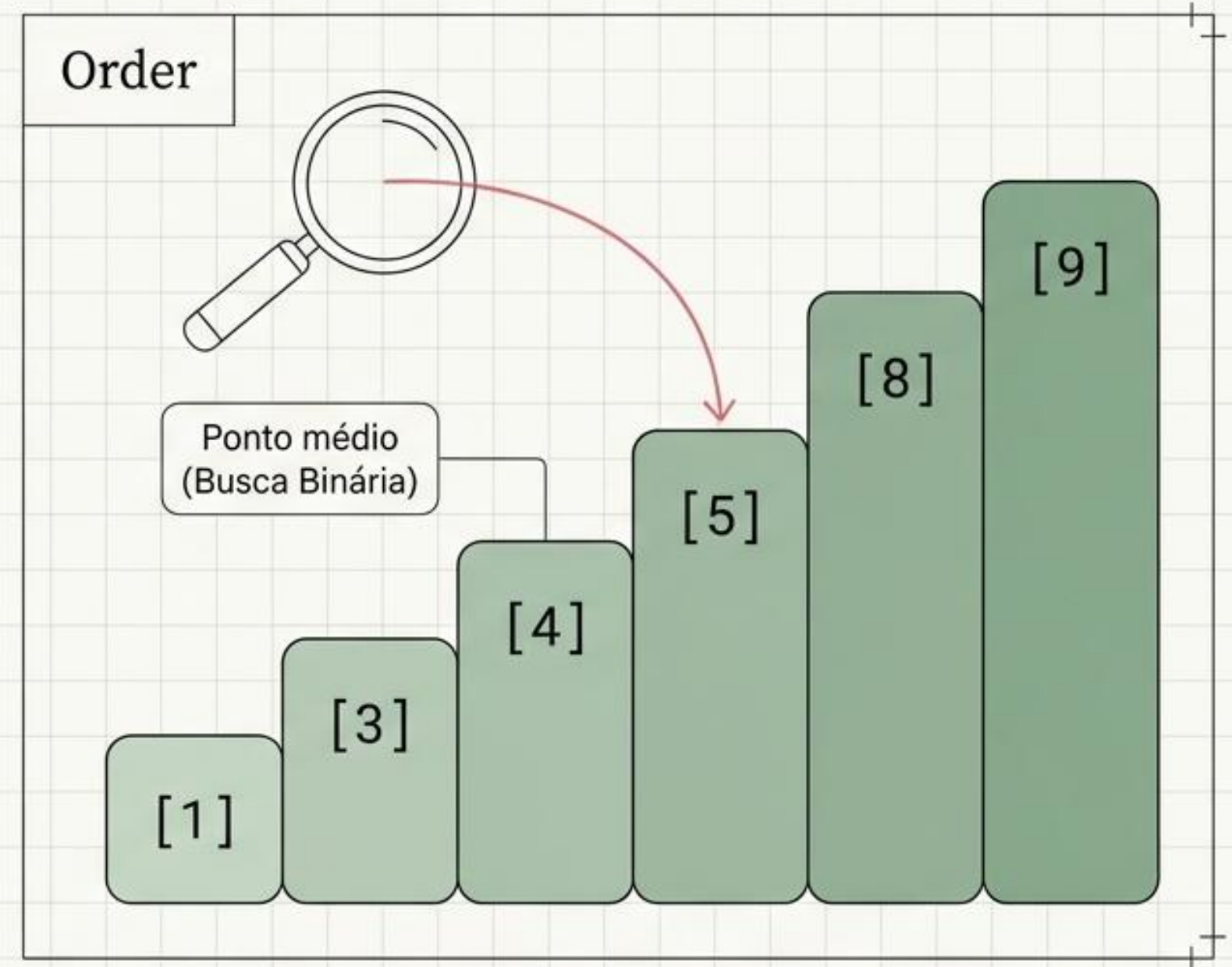
Dos algoritmos básicos às estratégias de ordenação eficientes.
A mecânica oculta de como o Python organiza o caos em escala.

O pré-requisito da velocidade

A busca binária é incrivelmente rápida, mas possui uma exigência absoluta: os dados precisam estar ordenados. Como transformamos uma lista desordenada como [8, 3, 5, 1, 9, 4] em [1, 3, 4, 5, 8, 9] sem usar atalhos como `sort()`?



Busca em dados desordenados = Lenta



Busca Binária ($O(\log n)$) = Ultra rápida

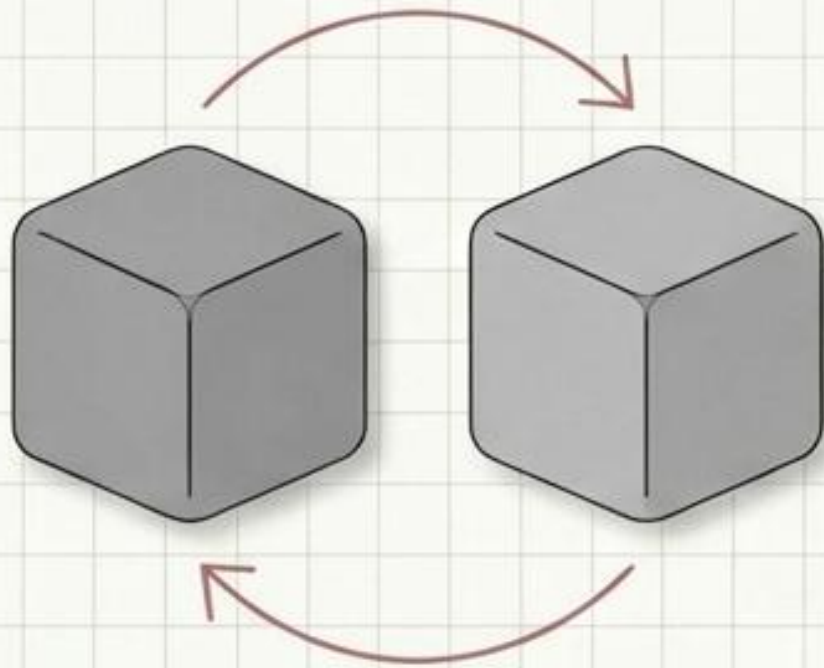
Ordenar é aplicar um critério

Algoritmos de ordenação reorganizam elementos seguindo uma regra matemática ou lógica estrita. O objetivo final é o mesmo; o que muda é o caminho para chegar lá.

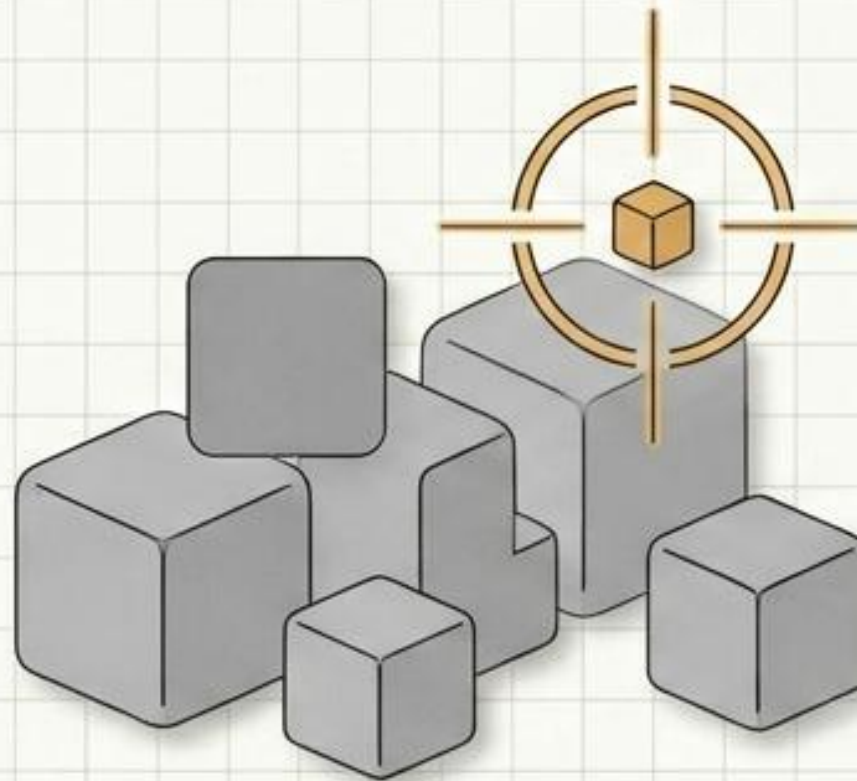


Três caminhos para o mesmo destino

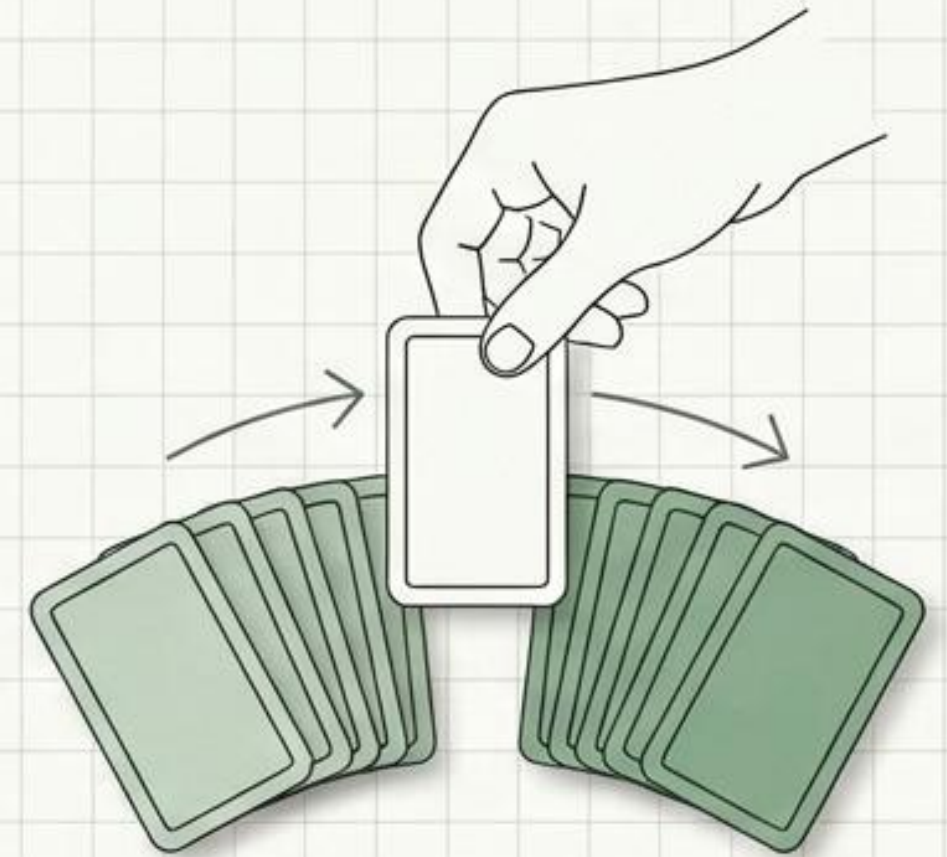
Para entender a otimização de sistemas reais, primeiro precisamos dominar as três mecânicas clássicas de ordenação. Todos chegam ao mesmo resultado, mas utilizam estratégias completamente diferentes.



Bubble Sort



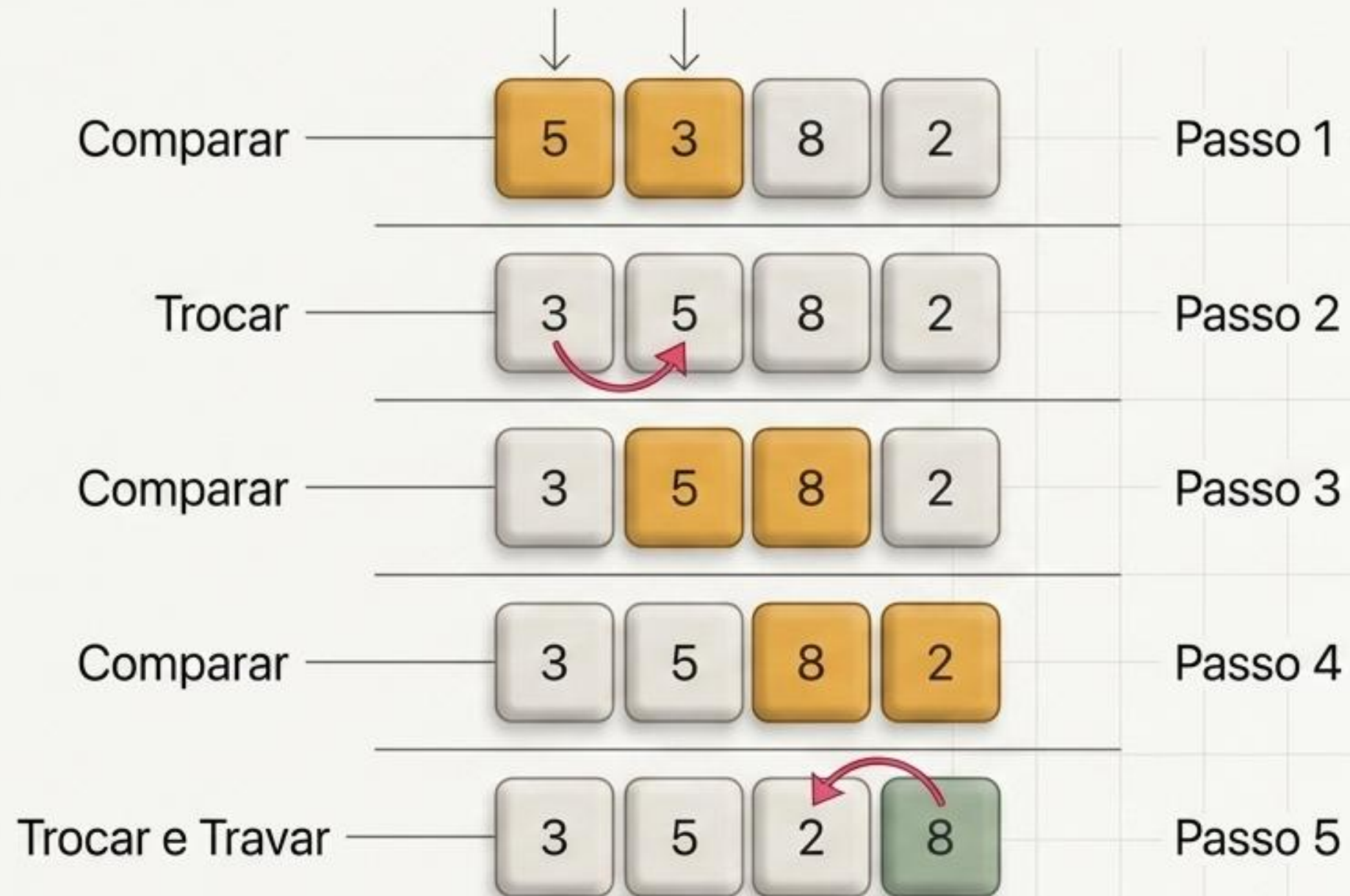
Selection Sort



Insertion Sort

Bubble Sort: O peso sobe à superfície

A ideia principal: comparar vizinhos. Se estiverem na ordem errada, eles trocam de lugar. Os maiores elementos borbulham rapidamente para o final.



Anatomia do Bubble Sort

O código reflete exatamente a ação física: varrer, comparar vizinhos e trocar.

```
def bubble_sort(lista):
```

```
    n = len(lista)
```

```
    for i in range(n):
```

```
        for j in range(n - 1 - i):
```

```
            if lista[j] > lista[j + 1]:
```

```
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
```

```
    return lista
```

1

Limita a varredura, ignorando a zona verde final.

2

O motor do Bubble: compara o elemento atual com seu vizinho imediato.

3

A ação: a troca de posições no mundo físico.

Selection Sort: O caçador de mínimos

Em vez de trocar vizinhos repetidamente, varremos a lista para encontrar o menor elemento absoluto e o colocamos diretamente em sua posição definitiva.



Anatomia do Selection Sort

Menos movimentações físicas que o Bubble, mas exige uma varredura completa da área não ordenada a cada ciclo.

```
def selection_sort(lista):  
    n = len(lista)  
    for i in range(n):  
        menor = i  
        for j in range(i + 1, n):  
            if lista[j] < lista[menor]:  
                menor = j  
        lista[i], lista[menor] = lista[menor], lista[i]  
    return lista
```

1

Assume temporariamente que o primeiro elemento da área não ordenada é o menor.

2

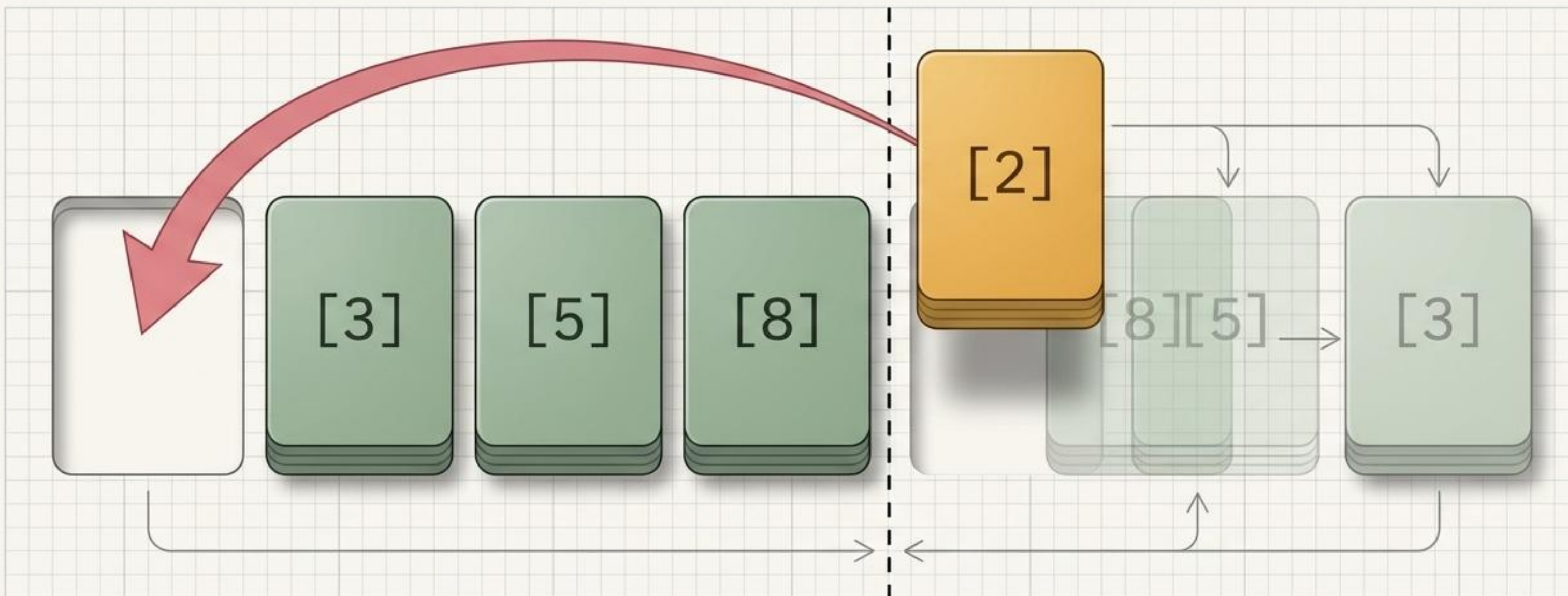
Atenção: A variável 'menor' guarda o índice (a coordenada), não o valor numérico em si.

3

Faz apenas uma troca por ciclo, movendo o menor encontrado para a zona segura.

Insertion Sort: Organizando o baralho

Mantém uma região à esquerda já ordenada. Cada novo elemento à direita é avaliado e inserido na posição exata dentro da zona ordenada, empurrando os maiores para o lado.



Anatomia do Insertion Sort

Não há trocas (swaps) puras aqui, mas sim deslocamentos. Valores maiores dão um passo à direita até surgir o espaço correto.

```
def insertion_sort(lista):  
    for i in range(1, len(lista)):  
        atual = lista[i]  
        j = i - 1  
        while j >= 0 and lista[j] > atual:  
            lista[j + 1] = lista[j]  
            j -= 1  
        lista[j + 1] = atual  
    return lista
```

1

A carta que acabamos de puxar do baralho.

2

O teste: enquanto a carta da mesa for maior que a minha carta atual...

3

...desloque a carta da mesa para a direita para abrir espaço.

Diagnóstico dos Algoritmos Manuais

Estratégias distintas, mas unidas por uma mesma limitação matemática crítica quando a lista cresce.

	 Bubble Sort	 Selection Sort	 Insertion Sort
Ação Principal	Trocar vizinhos	Achar o menor	Inserir no lugar certo
Mecânica Física	Empurra maiores para o fim	Constrói ordenado do início	Desloca para abrir espaço
Pior Caso	$O(n^2)$	$O(n^2)$	$O(n^2)$

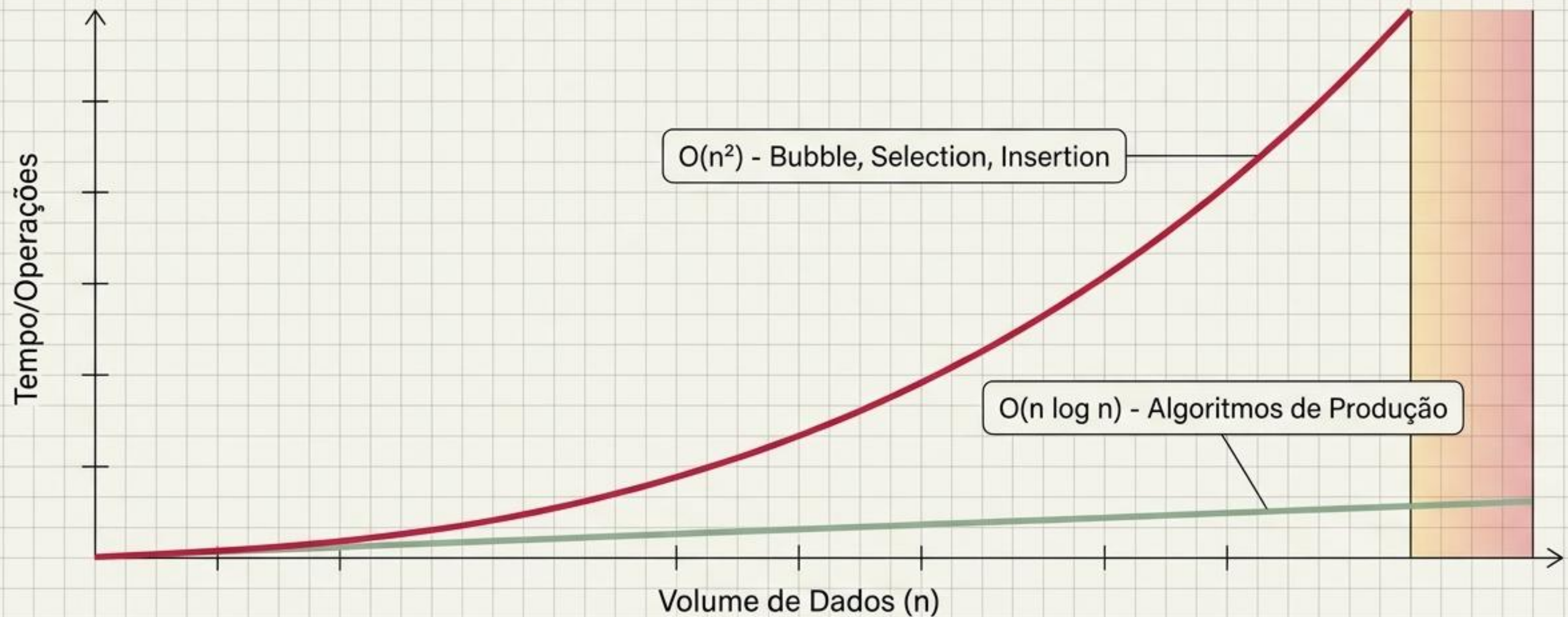
A Parede da Escalabilidade

Nos três algoritmos, o pior caso é $O(n^2)$. Isso significa que se dobrarmos o tamanho da entrada, o trabalho computacional cresce exponencialmente. Para listas pequenas, é invisível. Para milhões de registros, é catastrófico.

Tamanho da Lista (n)	Trabalho Máximo (n^2)
10	100
100	10.000
1.000	1.000.000
10.000	100.000.000

A Armadilha Quadrática

A complexidade $O(n^2)$ dita que o custo dispara fora de controle. É por isso que não usamos Bubble Sort em bancos de dados de grande escala no mundo real.



O fator das condições iniciais

O estado inicial dos dados muda tudo. Algoritmos da mesma classe ($O(n^2)$) têm custos práticos diferentes dependendo se a lista está invertida ou quase ordenada. Comparações custam CPU; movimentações custam memória.

Pior Caso



Comparações



Movimentações



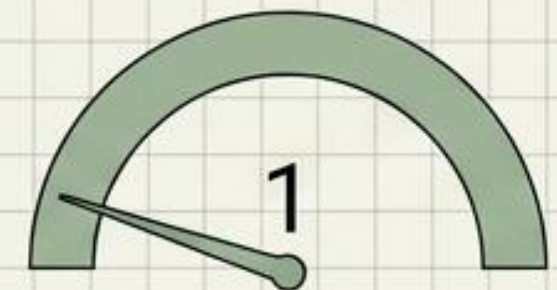
Quase Ordenada



Comparações



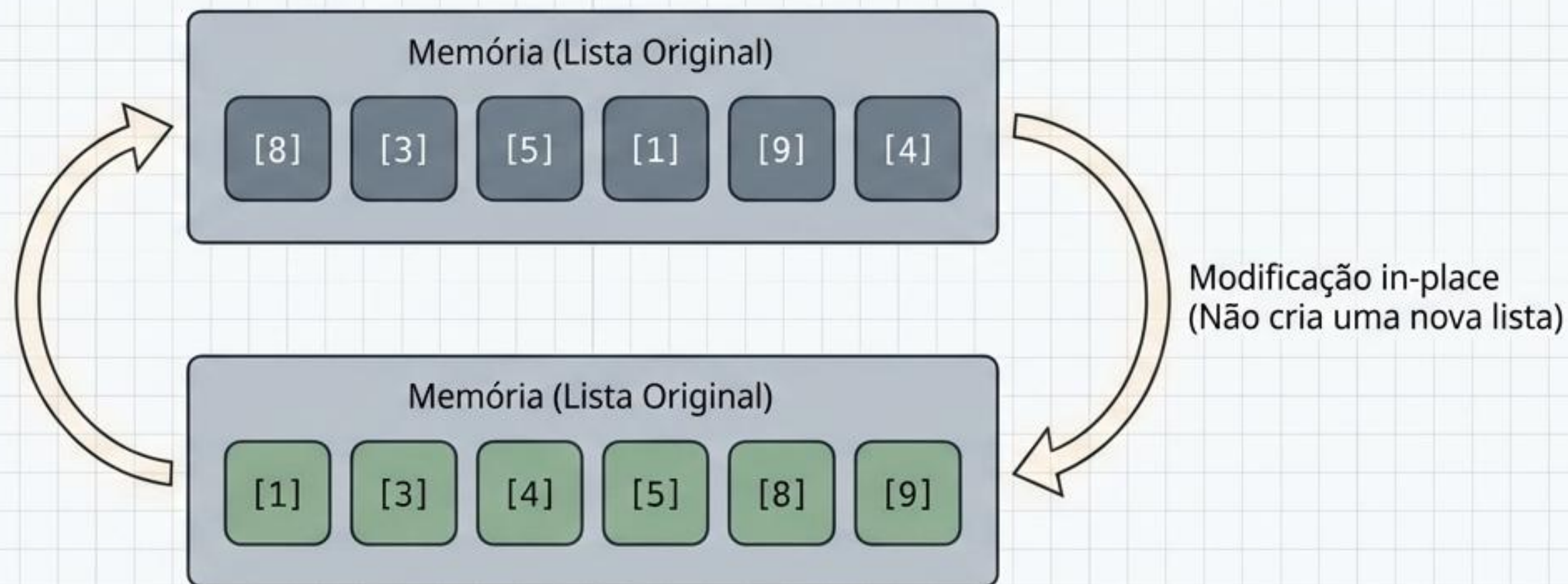
Movimentações



O padrão ouro do Python

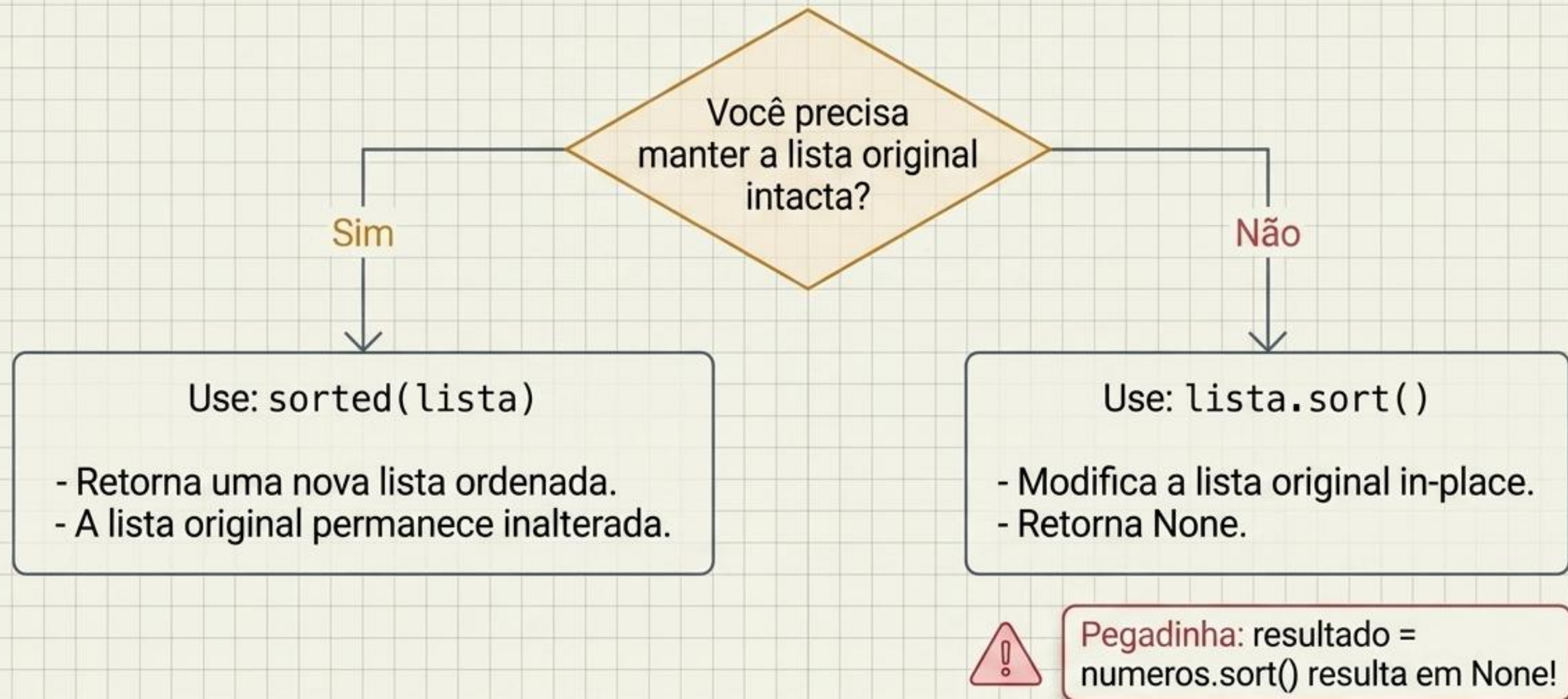
No mundo real, não reinventamos a roda. O método nativo **sort()** organiza os elementos **in-place** (modificando a própria lista na memória).

```
numeros = [8, 3, 5, 1, 9, 4]
numeros.sort()
print(numeros)
```



sort() vs sorted()

A escolha correta depende exclusivamente da sua necessidade de preservar os dados originais.



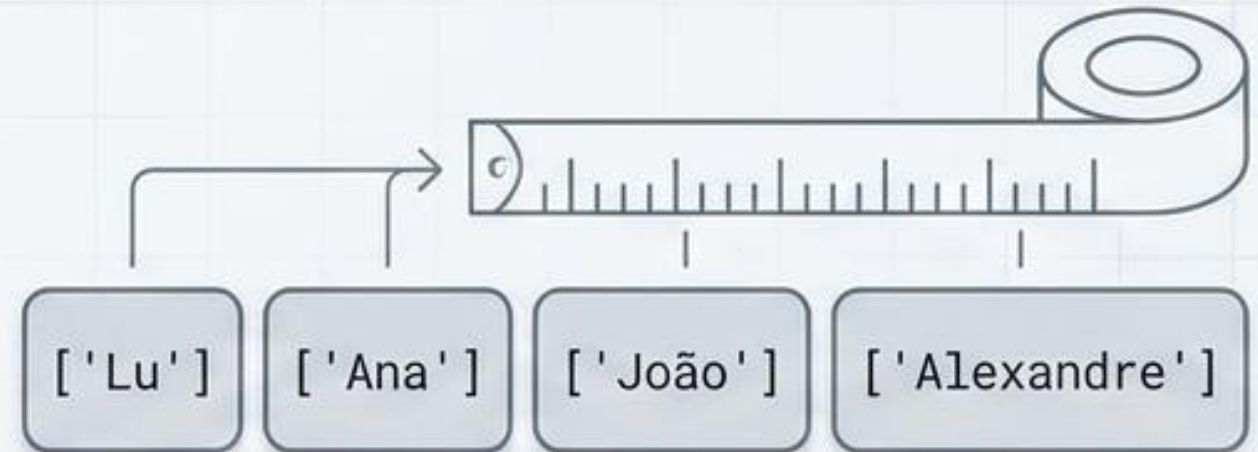
Flexibilidade em uma linha

A interface do Python permite inverter a ordem instantaneamente ou definir critérios complexos de comparação usando os parâmetros nativos.

```
numeros.sort(reverse=True)
```

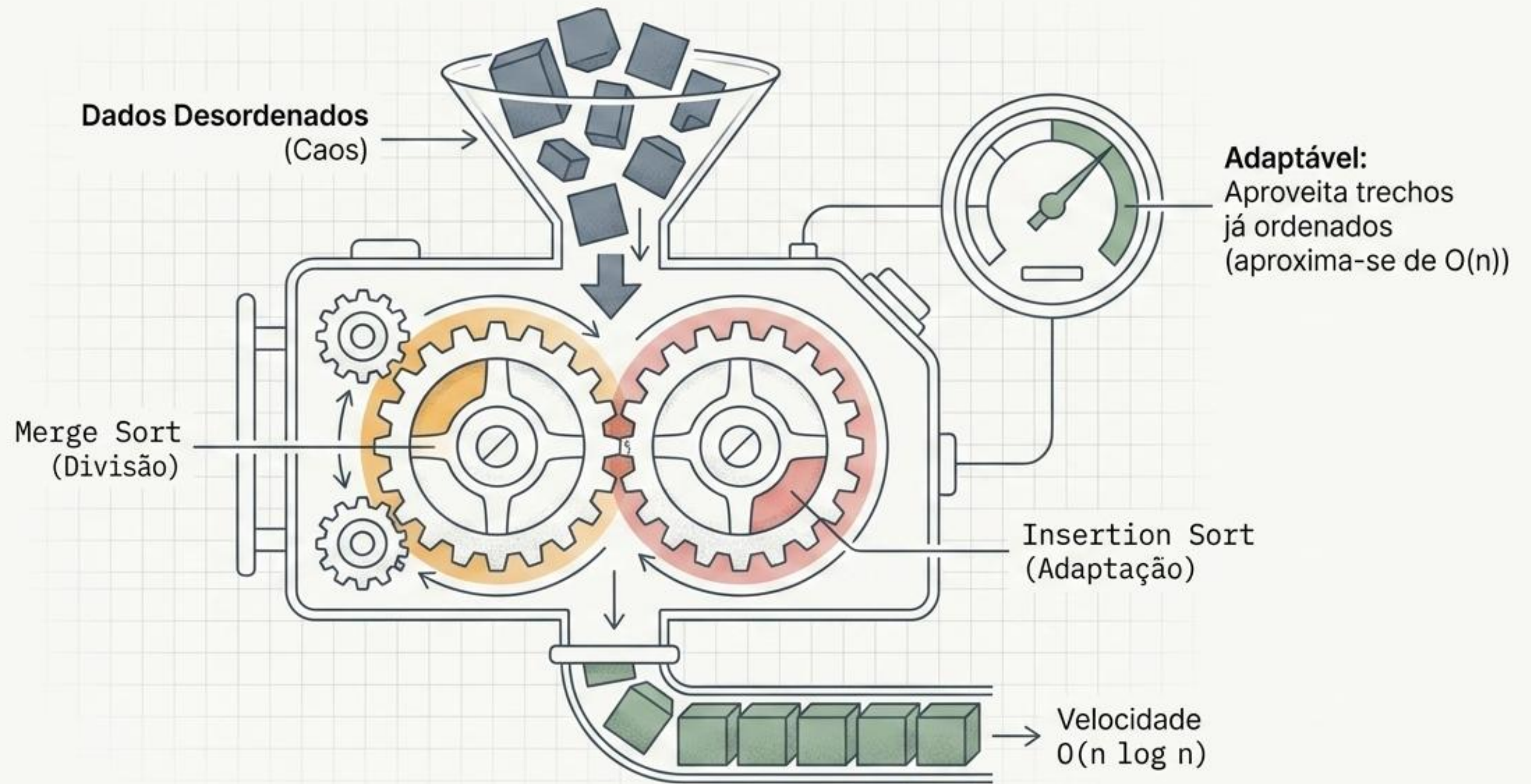


```
nomes = ['Ana', 'Alexandre', 'João', 'Lu']  
nomes.sort(key=len)
```



O Motor Oculto: Timsort

Quando você chama `.sort()`, o Python usa o Timsort, um algoritmo híbrido altamente otimizado que atinge $O(n \log n)$ e se adapta brilhantemente a dados parcialmente ordenados.



Código de Aula vs Código de Produção

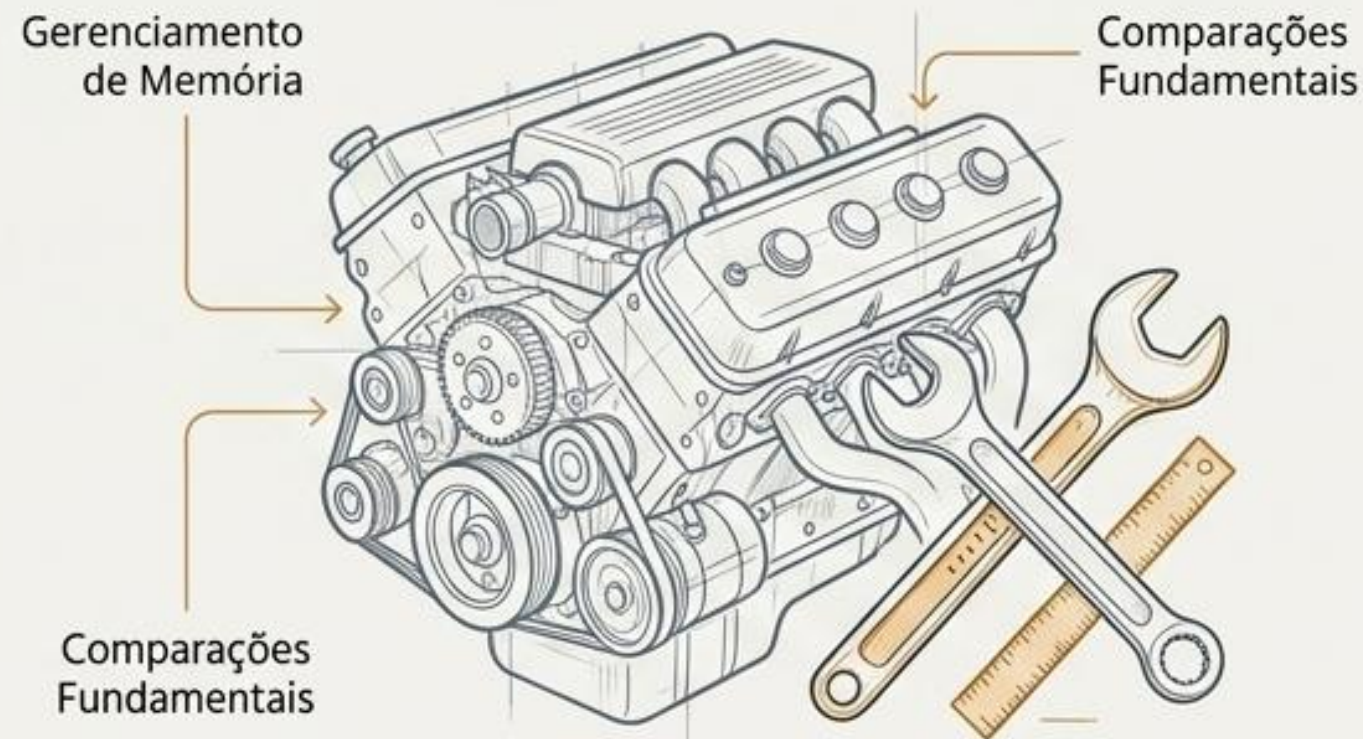
Algoritmos básicos são essenciais para o domínio lógico, mas ferramentas nativas são construídas para lidar com a escala do mundo real.

	Complexidade (Pior Caso)	Motor Principal	Objetivo de Uso
Algoritmos Manuais (Bubble/Sel/Ins)	$O(n^2)$	Código manual (Trocadas simples)	Aprendizado & Lógica
Python Nativo (<code>sort()</code>)	$O(n \log n)$	Timsort (Híbrido otimizado)	Produção & Escalabilidade

Por que aprender a criar o motor?

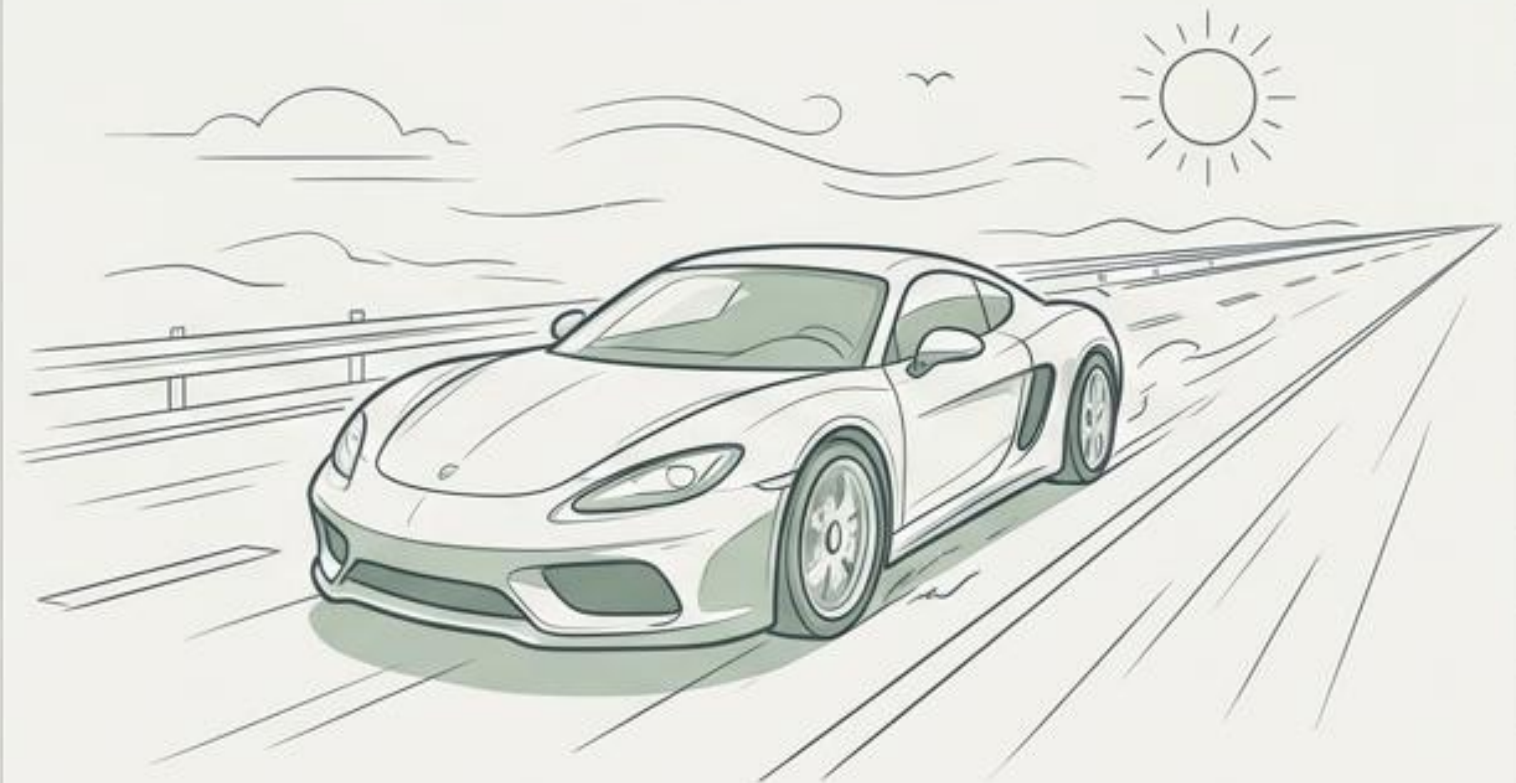
Se o Python já tem o `sort()`, por que estudar algoritmos lentos? Porque o `sort()` ensina a dirigir. Estudar movimentações e comparações ensina a construir o motor.

Engenharia (A Lógica base)



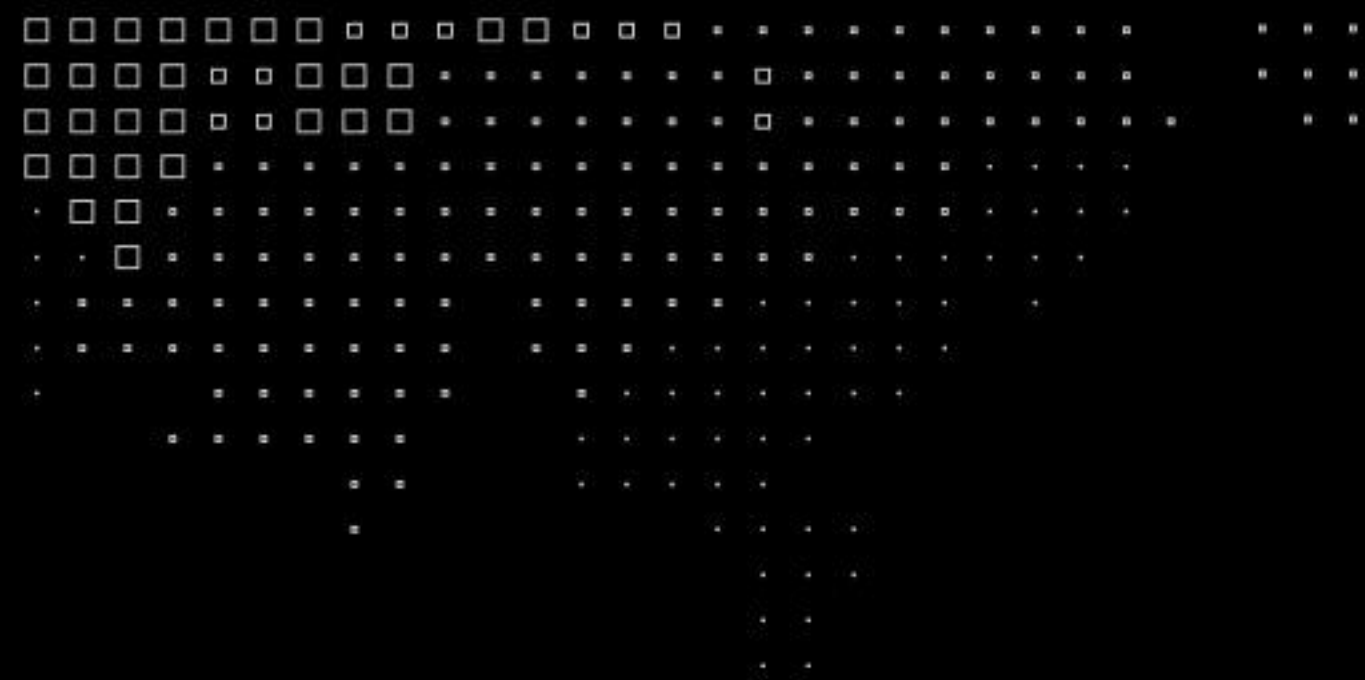
A base te dá a visão de raio-X sobre como as listas e a memória realmente se comportam.

Velocidade (A Ferramenta)



A ferramenta pronta te dá a escala necessária para o trabalho do dia a dia.

É como aprender multiplicação antes de usar uma calculadora.



Copyright © **2026** Prof. Álvaro Gonçalves

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor Álvaro Gonçalves.

